



REINFORCEMENT LEARNING

AULA 1



Prof. Gian Carlo Brustolin



CONVERSA INICIAL

Nesta etapa, vamos apresentar um apanhado geral de inteligência artificial (IA) envolvendo alguns conceitos básicos de nomenclatura e de taxonomia dessa técnica computacional importante. Iniciaremos trabalhando conceitos elementares e evoluiremos até o entendimento de técnicas básicas de treinamento e rudimentos de aprendizagem de máquina (*machine learning*), interpretação de parâmetros por máquina, *deep-learning* e *reinforcement learning* (RL).

Os estudos da aprendizagem em sistemas de IA sedimentou-se por duas metodologias clássicas de treinamento, de forma supervisionada e não supervisionada. Ambos os métodos se mostraram eficientes em meios determinísticos, mas não resolvem o problema de um sistema operando em um ambiente instável. Sistemas de RL, entretanto, enfrentam essa questão nativamente, o que justifica darmos destaque a essa técnica na conclusão desta etapa.

Vamos, então, iniciar nosso estudo introdutório. Recomendamos ao leitor que recorra à literatura indicada ao final deste capítulo para aprofundar os temas essenciais que doravante trataremos, de forma a estruturar uma base sólida para os temas específicos de IA a serem estudados futuramente.

TEMA 1 – CONCEITOS DE IA

Apesar de muito falarmos sobre IA, não há um conceito estabelecido sobre o tema. De fato, a própria definição de inteligência é lacunosa. A melhor aproximação que temos para uma máquina inteligente foi proposta por Alan Turing em 1950 (Norvig, 2013, p.1). O conhecido Teste de Turing permite avaliar se uma máquina computacional se comporta de forma inteligente, mesmo que não saibamos definir precisamente o que é inteligência.

As pesquisas em torno da IA acabaram por abandonar essa aproximação filosófica, tornando-se mais pragmáticas, no sentido de estudar simplesmente os princípios básicos da inteligência e sua reprodução sintética. O estudo da inteligência artificial, atualmente, “abrange lógica, probabilidade e matemática do contínuo, além de percepção, raciocínio, aprendizado e ação” (Norvig, 2013, p. 6).



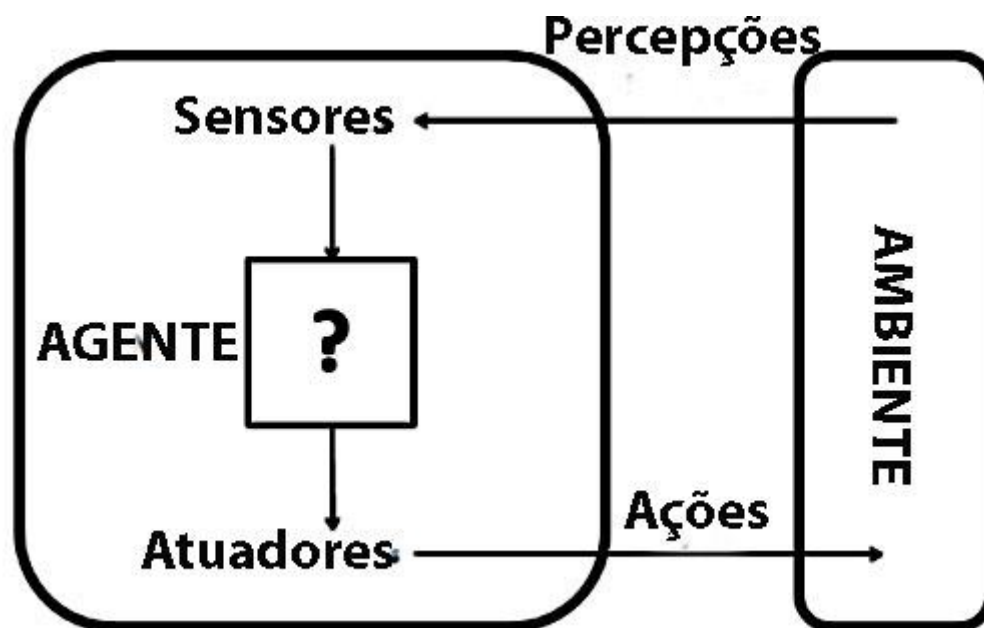
Dessa forma, quando iniciamos qualquer estudo sobre esse tema, devemos esperar a presença de muito mais conceitos matemáticos e estatísticos do que se imaginaria a princípio.

De qualquer forma, antes de mergulharmos nessas questões, devemos estabelecer uma nomenclatura comum, própria da disciplina de IA.

1.1 Agente inteligente

O conceito de agente inteligente é basilar e sempre o primeiro a ser estabelecido. Basicamente podemos afirmar que “um agente é tudo o que pode ser considerado capaz de perceber seu ambiente por meio de sensores e de agir sobre esse ambiente por intermédio de atuadores” (Norvig, 2013, p. 30). A Figura 1 ilustra esse conceito.

Figura 1 – Agente Inteligente



Fonte: Norvig, 2013, p. 31.

Neste ponto convém comentarmos sobre a distinção entre *agentes inteligentes reativos* e aqueles *baseados em aprendizagem*. Embora ambos sejam considerados agentes inteligentes, no primeiro caso todas as reações do agente são plenamente determinadas pelo seu criador, por meio da política (que examinaremos em seguida). O agente baseado em aprendizagem, por outro



lado, é capaz de aperfeiçoar a política, conforme se relaciona com o meio. Esses agentes são então capazes do que denominamos, atualmente, *aprendizagem de máquina* ou *machine learning*, em inglês. Observe que a ausência de aprendizagem não descaracteriza o agente reativo como inteligente (Faceli et al., 2021, p. 1).

1.2 Meio

O meio é o ambiente externo ao agente. A princípio, esse meio pode ser controlado ou não, tanto do ponto de vista de sua geografia quanto das possíveis aleatoriedades temporais. Um meio estável e controlado é dito *determinístico* já um meio que evolui no tempo é dito *estocástico*. Se o futuro do meio for completamente determinável pela percepção atual e pela ação do agente, será determinístico; caso contrário, ele é estocástico. Voltaremos a essa distinção quando tratarmos de RL, ainda neste capítulo.

Existem *meios padrão* ou *benchmarks*, criados para permitir o teste do comportamento do agente. Esses *benchmarks* são especialmente importantes para RL, como veremos um pouco mais à frente em nossos estudos.

O meio pode ser *integralmente observável*, em função da capacidade dos sensores de acessarem o estado do ambiente externo continuamente. A ausência de percepção de determinado dado, ruído e imprecisão de leitura dos sensores tornam o *meio parcialmente observável*. Isto não é, necessariamente, uma característica negativa: mesmo um agente desprovido de sensores (*ambiente inobservável*) pode realizar ações que dele se necessita (Medeiros, 2007).

A representação do meio para o agente, ou seja, a forma como o agente vê o meio é dita *modelo do meio*. Um agente pode aprender com base em modelos anteriores, tomando decisões com base no histórico de ações anteriores, ou pode tomar decisões sem este vínculo.



1.3 Ação

São as atuações do agente sobre o meio, como se observa na Figura 1.

1.4 Política e função do agente

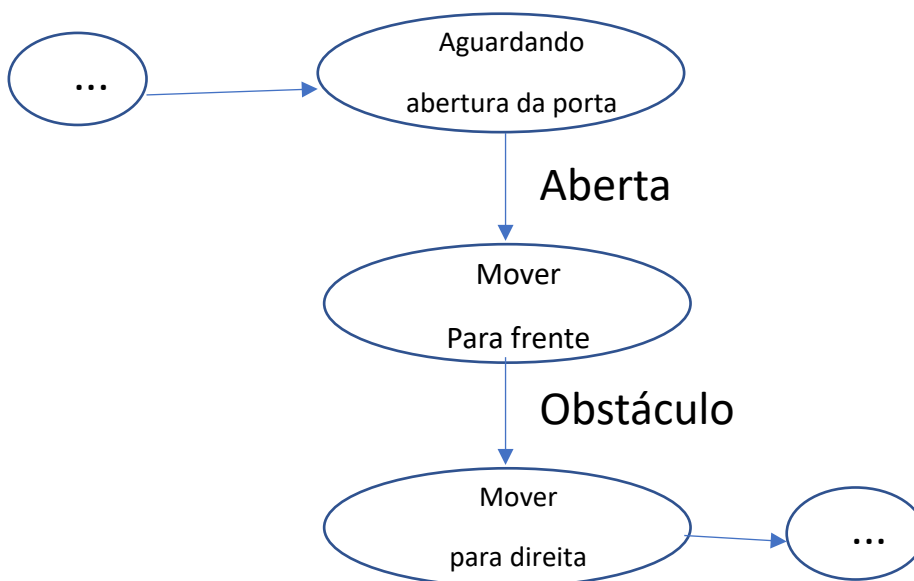
Quando uma agente se relaciona com o meio, realizando ações, é possível elaborar uma lista de todas as percepções e ações de resposta. Essa lista é uma caracterização externa do agente, conhecida como *função do agente*. Se observarmos o agente internamente, o que de fato ocorre é a implementação de um programa ou política do agente.

Norvig (2013, p. 31) enfatiza a importância de distinguirmos o programa (ou política) e a função de agente, que “é uma descrição matemática abstrata; (ao passo que) o programa do agente é uma implementação concreta, executada em um sistema físico”.

1.5 Estado

Basicamente o estado do agente designa sua relação atual com o meio, ou seja, localiza o agente em um determinado ponto de seu programa. Em um grafo que descreve o programa de determinado agente, cada nó constitui um estado do agente.

Figura 2 – Exemplo de estados de agente descritos por grafo





Outro conceito associado ao de estado é o de *valor do estado*. Bastante usado em RL refere-se a uma avaliação da utilidade de se manter em dado estado, ou mesmo de escolher se mover para dado estado, em relação à realização do objetivo final do agente.

TEMA 2 – ABORDAGENS DE IA

As pesquisas em torno de IA não têm um rumo único. Alguns pesquisadores preferem implementar técnicas de IA tendo por base o processo mecânico de raciocínio, até onde compreendemos, do cérebro humano. Outros preferem uma abordagem mais pragmática, baseando-se em um algoritmo de enfrentamento do problema e buscando a parametrização matemática das variáveis do problema. Um terceiro grupo estuda as formas de enfrentamento, pela natureza, dos problemas evolutivos, ou seja, busca reproduzir os algoritmos naturais presentes no universo e que resolvem problemas de altíssima complexidade (a exemplo do equilíbrio gravitacional de uma galáxia composta por milhões de astros).

A escolha da abordagem direciona e, eventualmente, limita as formas de aprendizagem elegíveis, embora, como já enfatizamos acima, o fato de existir ou não processos de aprendizagem não descaracteriza o comportamento inteligente de agentes cujas políticas estejam vinculadas a uma ou outra das abordagens de IA.

Vamos examinar superficialmente as duas abordagens mais frequentes de IA para que em breve possamos relacionar a elas as respectivas possibilidades de aprendizagem. Aproveitaremos essa abordagem clássica para também definirmos o entendimento atual de ML.

2.1 IA simbólica

Se acompanharmos a história do desenvolvimento de IA, iniciada com Turing no final da segunda guerra mundial, veremos que a primeira aproximação, ou tentativa de obter uma máquina inteligente baseou-se nos esforços de reprodução parcial da mente humana. Essa aproximação foi rapidamente



abandonada, principalmente em função da inexistência de capacidade computacional para processar as sinapses neurais artificiais.

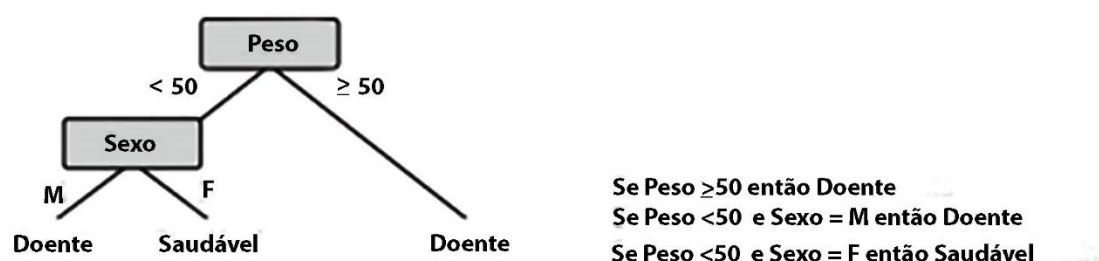
O modelo então adotado, que persiste operacional ainda hoje, foi o uso de algoritmos de solução de problemas dedicados, chamados de *sistemas especialistas* ou *sistema baseados em conhecimento*. Nesses sistemas, o projetista cria um *motor de inferência*, um algoritmo que toma as decisões baseado em regras *se-então*. Para que o conhecimento de especialistas possa ser agregado ao sistema, o método mais usado é o de entrevistas que buscam descobrir o algoritmo de tomada de decisões (Faceli et al., 2021).

Inicialmente os sistemas especialistas tinham inteligência limitada ao inserido pelo criador no momento da codificação. Técnicas de ML, entretanto, podem ser agregadas a esses sistemas, permitindo certa evolução. As limitações dessa técnica levaram ao gradual abandono desse tipo de sistema.

2.2 ML

Como comentamos, as técnicas de ML surgiram como incrementos aos métodos clássicos de IA, mas hoje são consideradas um dos ramos de IA. ML depende, entretanto, de um sistema de suporte. Dessa forma, parte das técnicas simbólicas receberam um novo alento com a implementação de algoritmos genéricos de busca, a exemplo de árvores decisão e técnicas de regras de decisão. A Figura 3 exemplifica esses modelos.

Figura 3 – Árvores de decisão e conjunto de regras



Fonte: Faceli et al., 2021, p. 5.

Algoritmos de ML são capazes de criar uma árvore de busca ou um conjunto de regras a partir de *datasets* tratados previamente. Os algoritmos



varrem as árvores ou regras, segundo dado viés de busca, na tentativa de obter capacidade de predição e generalização sobre o conjunto de dados original.

As técnicas de ML se assemelham a métodos de análise estatística descritiva ou preditiva, conforme a finalidade a que se destinam. Essa semelhança se vê enfatizada, quando nas aplicações de ML, utilizamos técnicas e conceitos estatísticos de análise. De fato, podemos conceituar o aprendizado de máquina, aproximadamente, como estatística computacional aplicada.

A popularidade dessa aproximação cresceu substancialmente com a disponibilidade de bibliotecas de algoritmos, capazes de realizar a descrição ou predição de fenômenos com base em *datasets* com certa facilidade. Linguagens de programação populares, como o Python, se tornaram conhecidas como boas opções para criação de aplicações de IA justamente pela existência de bibliotecas, de acesso gratuito, com boa variedade de algoritmos estatísticos, de fácil aplicação.

As técnicas clássicas de ML encontram seu limite justamente na preparação dos *datasets*. A grande maioria dos algoritmos de aprendizagem de máquina são classificadores estatísticos e precisam receber um vetor de variáveis específico. Os dados, em sua forma natural ou pura, de maneira geral, não podem ser diretamente oferecidos ao algoritmo de ML. A tarefa de transformar os dados brutos em um vetor representativo que permita ao algoritmo de ML detectar e classificar as entradas é frequentemente um profuso trabalho de engenharia de dados.

Neste ponto certamente você está se perguntando se não seria possível engenho um algoritmo capaz de transformar os dados brutos no vetor de características classificáveis consumível pelo sistema de ML. A resposta a essa pergunta são os métodos de *representation learning* ou *feature learning* (FL), os quais permitem a máquina consumir dados naturais e descobrir então as representações necessárias para o algoritmo classificador de ML. Técnicas de *deep-learning* (DL) são, de fato, evoluções de FL nas quais são admitidos vários níveis de representação. Podemos entender o algoritmo de DL como a utilização de uma pilha de métodos de FL, obtendo-se, a cada passo na pilha, um maior nível de abstração (Lecun; Bengio; Hinton, 2015). O interessante desse procedimento que estabelece as camadas de representação é que ele é



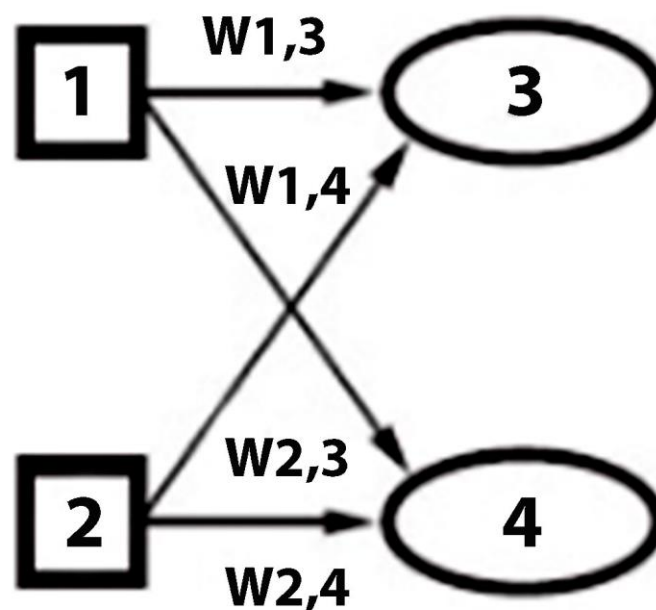
aprendido das características dos dados brutos por um algoritmo padrão de aprendizado de máquina e não envolve a decisão humana.

Agora estamos curiosos para saber como isso é possível. Procedimentos de DL são realizados por redes neurais, estudadas na clássica IA conexionista.

2.3 Neurociência computacional e a IA conexionista

Outra maneira de se enfrentar o problema da IA baseia-se na tentativa de simulação do processo biológico do raciocínio humano. A essa técnica se denominou *conexionista*, uma vez que entende o raciocínio pela óptica das conexões entre neurônios. Dito de outra forma, as pesquisas sobre o funcionamento do cérebro indicam que boa parte da memória e da capacidade de interpretação racional está ligada à forma como os neurônios se comunicam. Coube aos cientistas da computação modelarem matematicamente o neurônio com foco na capacidade controlada de conexão. Assim surgiram os modelos clássicos de neurônio que *memorizam* o aprendizado nos pesos sinápticos. A Figura 4 ilustra o controle dos pesos sinápticos ditos $w_{i,j}$ em uma pequena rede de neurônios.

Figura 4 – Pesos neurais em rede com 2 neurônios de entrada e dois de saída



Fonte: Norvig, 2013, p. 623.



O peso sináptico é um parâmetro livre da rede neural e permite *intensificar* ou *deprimir* a conexão entre dois neurônios. Para entender essa afirmação, vamos tomar a Figura 4. Suponha que o neurônio 1 é estimulado, externamente à rede produzindo como saída um determinado nível que chamaremos de y_1 . O neurônio 3 receberá a saída y_1 proveniente do neurônio 1, multiplicada pelo peso sináptico $w_{1,3}$; o mesmo se dará com o neurônio 4, que receberá como entrada $w_{1,4} * y_1$. Veja que a mesma saída y_1 estimulará os neurônios 3 e 4 de maneira diferente, ou seja, os pesos sinápticos w permitem regular a estimulação dos neurônios da rede.

Sedimentado este modelo neural básico, inicialmente apresentado em 1943 por McCulloch e Pitts, numerosas topologias de rede neural passaram a ser estudadas e a linha de investigação ganhou um nome mais pomposo, *neurociência computacional*. O problema passou a ser o entendimento de como se poderia agregar conhecimento a essas redes ou, dito de outra forma, como treiná-las para que memorizassem as informações.

Atualmente as técnicas de transferência de aprendizagem já estão suficientemente depuradas. Veremos a descrição de duas delas em nosso próximo tópico.

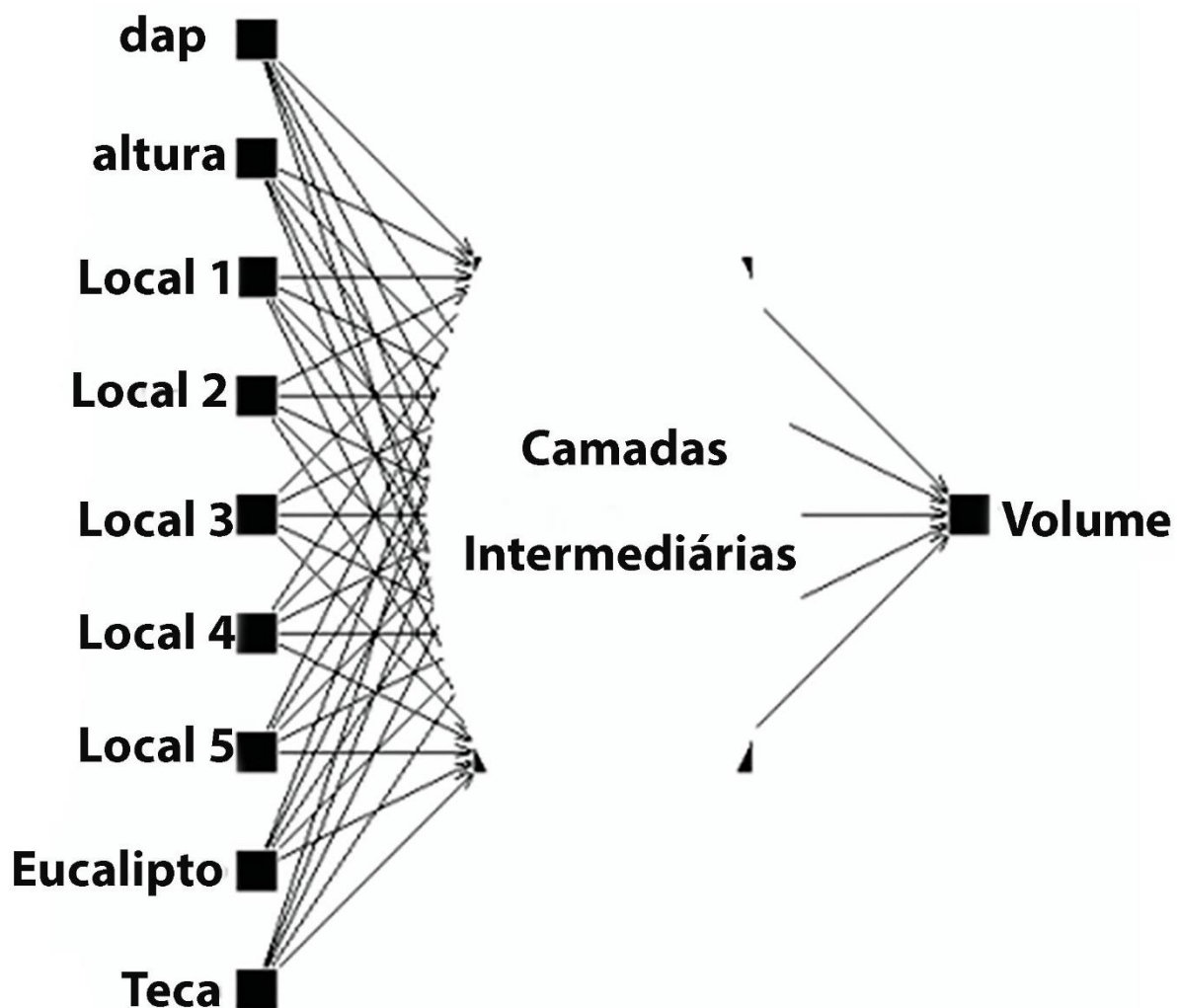
Norvig (2013, p. 629) nos adverte, entretanto, que a arquitetura da rede influencia diretamente na qualidade do aprendizado: “se escolhermos uma rede muito grande, ela será capaz de memorizar todos os exemplos, formando uma tabela grande de pesquisa, mas não generalizará bem, necessariamente, as entradas que nunca foram vistas antes”. Assim, as redes neurais também se aproximam de modelos estatísticos e estão sujeitas a superadaptações.

Inicialmente podemos pensar que o tratamento de um problema complexo necessitaria de uma rede neural artificial (RNA) com centenas ou milhares de neurônios, mas esse é um pensamento bastante equivocado.

No exemplo prático ilustrado pela Figura 5, uma RNA de 3 camadas com 15 neurônios foi capaz de prever o volume disponível de madeira em árvores de espécies diferentes. Esse cálculo pode ser feito, sem uso de algoritmos de IA, por equações volumétricas obtidas por regressão para cada estrato, material genético, espaçamento, regime de corte e idade. Essas equações, além de complexas, precisam ser atualizadas anualmente, gerando alta demanda de profissionais e recursos (Gorgens et al., 2009).



Figura 5 – RNA para previsão do volume de madeira em árvores



Fonte: Gorgens et al., 2009.

Como se percebe na ilustração, pequenas redes são capazes de aprender problemas complexos e apresentar a generalização necessária para que possamos utilizá-las como ferramentas de predição e análise.

Problemas tradução de representações com uso de algoritmos neurais de Deep-learning (DL), por outro lado, devem ser enfrentados por redes com múltiplas camadas ocultas. O número de neurônios pode não ser alto, embora normalmente maior que nas RNA de propósito geral, mas a quantidade de camadas ocultas é maior. Há alguma relação ainda não estabelecida, mas já conhecida, entre o número de camadas da rede e o nível de abstração possível



para o algoritmo de DL. O aprofundamento das redes, com o acréscimo de camadas intermediárias, incrementa substancialmente o problema do treinamento da rede. Em nosso próximo tópico veremos como é possível treinar uma rede neural e entenderemos melhor essa dificuldade aqui citada.

2.4 IA fraca e IA forte

Uma outra classificação, mais recente, dividiu a inteligência artificial em duas linhas distintas de pesquisa, uma que busca assemelhar o comportamento do agente inteligente ao humano, capacitando-o a enfrentar ambientes inusitados e tomando decisões hipoteticamente autônomas. A essa aproximação se designou *IA forte*.

Em contraposição a essa linha, teríamos a aproximação tradicional de IA, que quer dar a cada agente somente a inteligência necessária para enfrentar um dado tipo de problema. Todas as aplicações comerciais atuais seguem esse princípio, permitindo que um algoritmo enfrente um tipo de problema, adaptando-se a variações controladas no escopo de sua criação.

TEMA 3 – TEORIA DA APRENDIZAGEM E TREINAMENTO EM AMBIENTES DETERMINÍSTICOS

Um agente, para que adquira um comportamento inteligente, precisa adquirir memória, a qual pode ser inserida no agente diretamente pelo criador ou por um processo de aprendizagem. Há muitos tipos de aprendizagem. Neste momento, por exemplo, estamos exercitando um tipo de aprendizagem pelo estudo destes temas de IA. É possível aprender também pela dedução, ou seja, conhecidas várias teorias somos capazes de criar uma nova hipótese a partir deste conhecimento prévio.

O conhecimento empírico, aquele advindo da observação dos fenômenos externos, é outra fonte rica de aprendizagem. Do ponto de vista humano, a aprendizagem empírica se dá pela indução, associando a observação de fenômenos a uma teoria complexiva. A essa teoria poderíamos chamar de *generalização das observações*.



Do ponto de vista de IA, segundo Norvig (2013, p. 605), essa classe de aprendizagem é conhecida como *aprendizagem baseada em exemplos*. O interessante dessa classe é que “qualquer componente de um agente pode ser melhorado através da aprendizagem a partir dos dados (exemplos)”. Naturalmente esse tipo de aprendizagem depende da previsibilidade do ambiente, ou seja, que os eventos futuros repitam os resultados dos eventos passados. Esse tipo de meio é chamado de *determinístico*.

3.1 Treinamento de agentes

Antigos sistemas especialistas já possuíam as informações necessárias para a tomada de decisão em sua concepção, entretanto algoritmos de aprendizagem podem ser agregados a esses sistemas para que evoluam. Nesse caso, a eventual aprendizagem se dá posteriormente à entrada em operação. Redes neurais, por sua vez, são sujeitas a um treinamento prévio em relação à sua operação, embora, da mesma forma que no exemplo anterior, possam evoluir após sua entrada em operação.

Modelos de ML também aprendem em função de conjuntos de dados escolhidos para esse fim, chamados de *datasets*. Concluído esse aprendizado, o sistema entra em operação utilizando as estimativas obtidas no treinamento. Evoluções posteriores são possíveis, mas, novamente, como nos exemplos anteriores, dependem de algoritmo específico para esta evolução.

Percebemos então que o termo *aprendizado* pode ser aplicado tanto à fase de pré-operação (mais comum) quanto após a entrada em operação como forma de permitir a evolução do agente. Há, entretanto, modelos de treinamento que facultam o aprendizado nas duas fases, como no caso do RL. Este fato nos permite afirmar que o entendimento de RL depende do entendimento prévio dos métodos pré-operacionais de treinamento. O estudo dos parâmetros básicos de RL será objeto de nosso próximo tópico, ainda neste capítulo.

3.2 Métodos de treinamento de agentes

Seguindo ainda o que nos ensina Norvig (2013, p. 592), quatro são os fatores que vinculam o sucesso de uma técnica de aprendizagem computacional:



qual componente deve ser melhorado, o conhecimento prévio do agente, a representação dos dados e o *feedback* que está disponível para aprendizagem.

Quando mencionamos os *componentes do agente*, nos referimos àqueles previstos pela teoria clássica de IA, ou seja: mapeamento estado atual *versus* ação, algoritmo de dedução a partir das percepções do agente, informações da desejabilidade e utilidade dos estados e classes de estados cuja realização maximiza a utilidade do agente.

O *conhecimento prévio* se refere ao método que se utiliza para treinar o agente antes de sua operação, ou seja, de que forma preparamos o agente para que possa atuar no meio ou para que melhor absorva as informações de um novo aprendizado. Esse fator leva em conta, assim, a inserção de um viés de aprendizado. Em redes neurais, por exemplo, a arquitetura da rede pode torná-la mais eficiente para determinada tarefa, como o reconhecimento de padrões em imagens ou a detecção de anomalias em amostras. Pode-se dizer, então, que a escolha desta arquitetura enviesada o aprendizado da rede.

No que se refere à *representação dos dados*, é necessária uma compreensão dos dados que desejamos utilizar para treinar o agente. Dados podem ter dimensões ou características descritivas incompatíveis sem um tratamento prévio. Um bom exemplo para entendermos essa necessidade está na presença de dados de entrada em uma rede neural com dimensões muito distintas. Suponha que queiramos prever o ponto de corte de uma árvore. Os dados de entrada podem conter a altura da árvore em metros e a umidade média do solo em gramas de água por centímetro quadrado, por exemplo. Valores típicos seriam alguns metros no primeiro caso e algumas microgramas, no segundo. Se tentarmos treinar essa rede com entradas desse tipo, uma variação insignificante no parâmetro de altura teria grande impacto sobre a saída da rede, ao passo que mesmo se dobrando a umidade média do solo, a rede não apresentaria sensibilidade de resposta. Seguindo o mesmo caso, um dado qualitativo pode ser importante, por exemplo, a cor das folhas. Esse dado precisaria ser transformado em um parâmetro quantitativo para que a rede possa entendê-lo.

Finalmente, o *feedback* disponível em um meio define diretamente o tipo de aprendizagem que poderá ser utilizado. Por *feedback* entende-se a informação que o meio pode fornecer (previamente ou não) quanto ao resultado



da ação do agente inteligente. Vamos ao exemplo de nossa rede neural de previsão de corte de árvores. Suponha que ela nos indique que uma árvore inadequada deve ser cortada, podemos utilizar uma metodologia de aprendizagem para corrigir o erro médio dessa rede. Nesse caso, utilizamos o conhecimento de um par entradas-saída, ou seja, há um instrutor de treinamento, que *ensina a rede*.

Se não houver *feedback* algum disponível, a aprendizagem ainda é possível, porém com características bem particulares, que vamos discutir em nosso próximo tópico.

Um último tipo de *feedback* se dá quando precisamos que o agente entenda as reações de um meio mutante a suas ações. Nesse caso, mesmo que tenhamos pares entrada-saída, a saída, para uma dada entrada, se altera com o tempo. O agente deverá aprender a partir de um processo de recompensas e punições, medindo a adequação de sua ação frente ao meio em dado momento.

Antes de começarmos a estudar esses três tipos de aprendizagem, devemos entender algumas regras de aprendizagem de agentes inteligentes, seguindo o que nos ensina Haykin (2011, p. 76 e seguintes) e Copping (2010, p. 233 e seguintes). Não se pretende exaurir os algoritmos de aprendizagem, mas vamos apenas conhecer alguns para que seja possível ilustrar posteriormente as classes de aprendizagem.

3.3 Aprendizagem por correção de erro

A aprendizagem por correção de erro tem mais aplicação em sistemas conexionistas de IA (redes neurais) e por esse motivo focaremos nossos exemplos nesses modelos, mas talvez seja o método mais intuitivo de aprendizagem.

Trata-se de selecionar um conjunto de dados de entrada (x) cuja saída conheçamos. Inserimos os dados de entrada no algoritmo de IA e coletamos sua saída (y). Como a saída desejada (d) é conhecida podemos determinar se ocorreu erro (e).

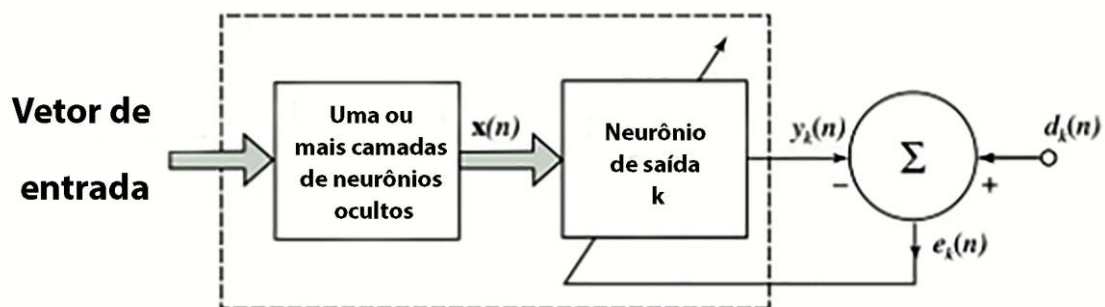
Tomando-se uma saída numérica (escalar) para o algoritmo, o erro poderia ser calculado como a diferença entre o valor desejado e o de saída, como na expressão abaixo, para o dado n :



$$e(n) = d(n) - y(n)$$

Aplicado às redes neurais temos o esquema descrito na Figura 6, onde analisamos a saída de um único neurônio em função de um conjunto de dados de entrada, em uma rede neural de camadas múltiplas qualquer.

Figura 6 – Treinamento por correção de erro em RNAs



Fonte: Haykin, 2011, p. 77.

Hipoteticamente, seria possível ajustar os parâmetros da rede para que o vetor de entrada $x(n)$ apresente como saída o valor $y(n)=d(n)$.

Esses parâmetros da RNA, chamados de *parâmetros livres da rede*, são os pesos sinápticos, que, como já estudamos, alteram os valores de saída-entrada no interior da rede, entre os neurônios, e o ajuste de bias de cada neurônio.

Podemos também imaginar que, das amostras oferecidas à rede, algumas produzirão erro e outras não. Assim, é útil medirmos o quanto a rede converge conforme ajustamos os parâmetros. Nesse sentido, como existem erros positivos e negativos, impedindo o cálculo da média como indicador de convergência, o cálculo da energia de erro, uma espécie de erro médio quadrático, nos fornece uma boa indicação da evolução da rede. Uma técnica bastante usual para a correção do erro médio da rede é a regra de Widrow-Hoff (ou delta).



3.4 Aprendizagem baseada em memória

Suponha que você tenha um conjunto finito de “n” dados de entrada, que chamaremos de *vetores de entrada* $x_i(n)$ para os quais conhecemos o resultado desejado $d(n)$. Voltando ao nosso algoritmo de previsão de corte de árvore, o vetor de entrada seria composto, por exemplo, de dados como altura, $x_1(n)$; umidade média do solo, $x_2(n)$ e coloração das folhas, $x_3(n)$. Para cada vetor composto destas três informações teríamos uma única saída binária, informando se devemos cortar a árvore, $d(n)=1$; ou não, $d(n)=0$.

Na aprendizagem baseada em memória, memorizaremos vários exemplos de entrada-saída classificados em dada ordem. Retomando ao corte de árvores, poderíamos ter, já convertendo a última variável de entrada (cor) em padrão numérico RGB, $\{[(3;0,0004;255,255,0),1]; [(3,5;0,0004;200,199,0),1]; [(1,5;0,0004;0,200,199),0]...\}$. Nesses exemplos, temos os dois primeiros conjuntos de dados com saída 1, ou seja, indicativo de corte, e o terceiro com saída 0, não indicando o corte. Veja que nessa aproximação de aprendizagem, todos os valores dos exemplos são verdadeiros, ou seja, neste exemplo, a saída “1” indicada de fato a necessidade de corte, enquanto a saída “0” reflete situações em que não se deve ainda cortar a árvore.

Ao colocarmos o algoritmo em operação, compararemos o valor de entrada (chamado de *vetor de teste*) com as amostras de memória e tomaremos a decisão mais próxima. Se, em nosso exemplo de corte de árvore, recebermos o seguinte vetor de teste:

$$x_1 = 2,9; x_2 = 0,0004 \text{ e } x_3 = 199,249,0 .$$

O algoritmo decidirá pela saída “1” ou “0” conforme a entrada de teste mais se aproximar de uma amostra da memória. Neste exemplo que estamos estudando, com os três conjuntos de dados que apresentamos, a decisão seria pelo corte da árvore. Essa resposta é intuitiva, pela mera comparação visual das amostras, mas podemos criar um algoritmo determinístico para a tomada de decisão.

Segundo Haykin (2019, p. 79), todos os algoritmos de aprendizagem baseada em memória envolvem uma boa escolha do critério para definir quando a vizinhança será suficientemente próxima para que seja feita a mesma escolha

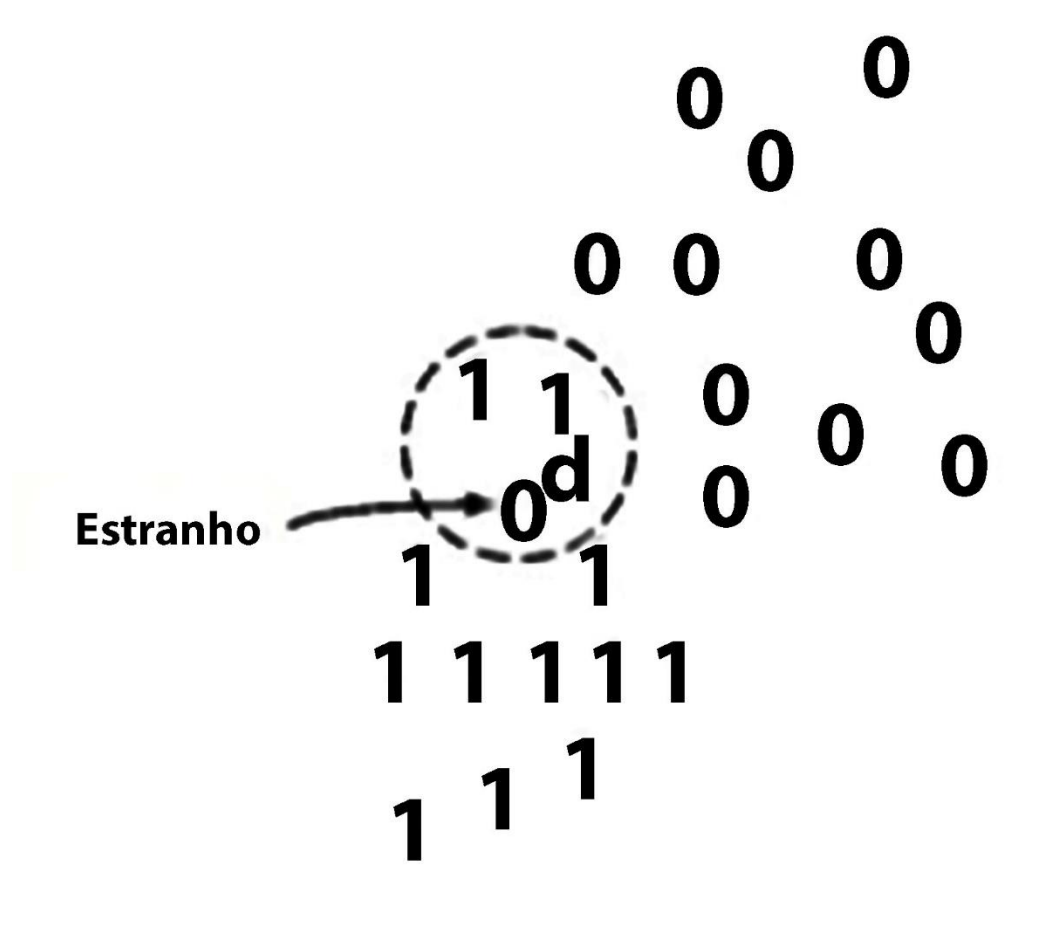


do vizinho. Outro fator importante é a regra para escolha (e eventualmente incremento) do conjunto de amostras de memória.

A princípio, a menor distância identifica o *vizinho* do qual emprestaremos a decisão, mas definir a menor distância, quando o vetor de entrada é composto por vários elementos, é uma tarefa um pouco mais complexa.

Outra aproximação que pode melhorar ainda mais este algoritmo é o chamado *classificador* pelos “k” vizinhos mais próximos. De acordo com esse método, não escolhemos apenas o vizinho mais próximo, mas os “k” mais próximos e, em seguida, tomamos a decisão por maioria. A Figura 7 ilustra o procedimento. Nesta representação, o ponto “d” é a localização do vetor de teste e os pontos “0s” e “1s”, as saídas das amostras de memória.

Figura 7 – Método dos “k” vizinhos



Fonte: Haykin, 2011, p. 80.



3.5 Aprendizagem por árvores de decisão

A aprendizagem por árvores de decisão é uma das formas mais antigas de aprendizagem, idealizada durante o reinado dos sistemas especialistas. Algoritmos de ML, que criam a árvore automaticamente com base em *datasets*, deram nova leitura a essa técnica.

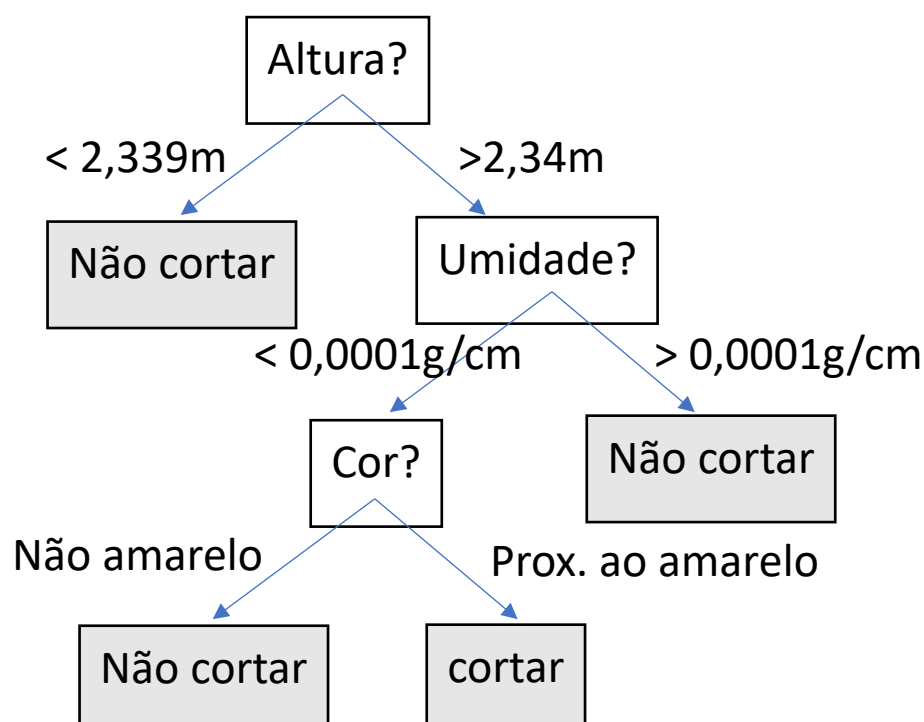
No algoritmo de árvore, um vetor de entrada encontrará uma saída correspondente em função de uma série de testes de hipóteses ocorridos no interior da árvore. Os nós da árvore correspondem a esses testes sobre um dos valores do vetor de entrada.

Vamos recorrer novamente a nosso sistema para recomendação de corte de árvore. Podemos criar uma variação aceitável para cada elemento do vetor, por exemplo:

- x – altura da árvore para corte acima de 2,34m;
- x – unidade média para permitir o corte abaixo de 0,0001g/cm;
- x – cor da folha próxima ao amarelo $R > 180$, $G > 180$ e $B < 30$.

Com esses dados, podemos tomar a decisão em relação a uma árvore. Se escolhermos a raiz como sendo a altura, podemos montar a árvore como na Figura 8:

Figura 8 – Árvore de decisão





Os algoritmos de árvore de decisão são bastante eficientes para determinados tipos de problemas, mas há problemas que não podem ser enfrentados por esse método. Em uma árvore de decisão, é necessário escolher a ordem das perguntas (ditos atributos) segundo sua relevância, ou seja, é necessário escolher o atributo mais importante como raiz. Fato é que nem sempre há atributos distintamente significativos e nesses casos as árvores podem se tornar excessivamente extensas ou mesmo inconclusivas (Norvig, 2013, p. 611).

Algumas linguagens de programação que possuem bibliotecas de ML identificam a melhor árvore de decisão levando em conta um conjunto de dados com uma parametrização mínima. O melhor exemplo é a biblioteca *Scikit-learn* do Python, que examina um *dataset* pré-processado convenientemente e nos retorna à árvore de decisão ótima. O processamento prévio é feito, de maneira geral, com o uso de outras bibliotecas auxiliares, como Pandas e NumPy.

3.6 Aprendizagem competitiva e hebbiana

Essa forma de aprendizagem utiliza a propriedade de auto-organização estatística dos fenômenos naturais, fato que melhor discutiremos quando abordarmos o treinamento não supervisionado. Utilizada principalmente em IA conexionista, essa técnica se baseia em uma particularidade da aprendizagem humana.

O neuropsicólogo Hebb, na metade do século passado, observou que, no cérebro humano, quando um dado neurônio N, próximo ao neurônio P, repetidamente provoca a reação deste neurônio P, a proximidade físico-química entre esses neurônios cresce, potencializando ainda mais essa reação. Esse fato, segundo o mesmo pesquisador, está relacionado a determinados tipos de aprendizagem humana (Haykin, 2011, p. 80).

Diante dessa constatação criou-se uma técnica de treinamento neural que reproduz este fenômeno natural, corrigindo-se os parâmetros livres da rede, conforme a saída de um neurônio se vê estimulada. Percebido o estímulo de saída, o peso das sinapses conectadas a essa saída é majorado.



Procedendo-se dessa forma, se um conjunto suficientemente grande de dados for passar por um sistema neural, os neurônios tenderão a reagir a esses estímulos segundo um determinado padrão, e a rede treinará a si mesma para reconhecê-los.

TEMA 4 – TREINAMENTO SUPERVISIONADO *VERSUS* AUTO-ORGANIZADO

Vamos, em seguida, estudar de maneira bastante resumida esses dois modelos e, como de hábito, sugerimos ao leitor complementar o estudo na literatura indicada ao final deste capítulo. Estes não são os únicos modelos possíveis, existe ainda um tipo de treinamento que pode ser dito não supervisionado ou parcialmente supervisionado, o treinamento por reforço.

A literatura científica por vezes prefere se referir ao treinamento supervisionado como “aprendizado com um professor” (Haykin, 2011, p. 88), uma vez que algoritmos de RL podem ser considerados supervisionados pelo meio, mas sem um *professor* exatamente.

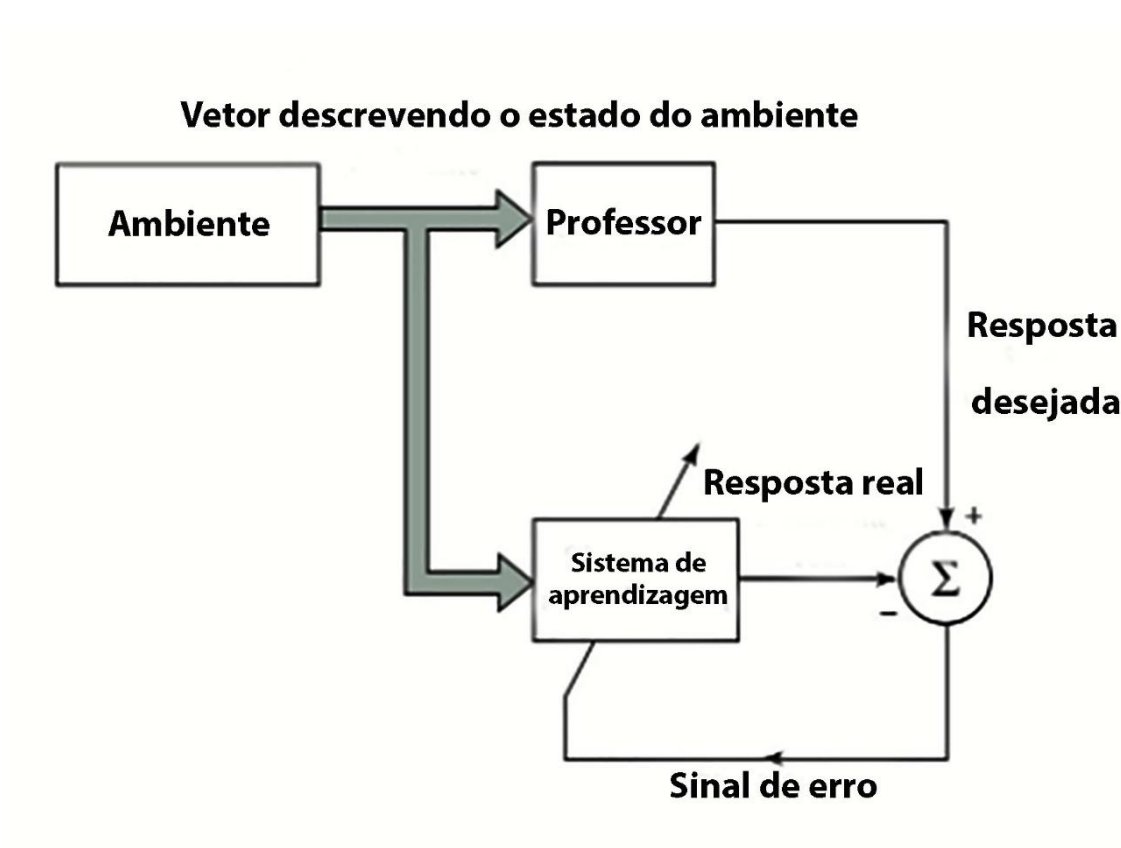
O treinamento por reforço será abordado, em corte específico, em nosso próximo tópico, já que seu modelo de aprendizagem é bastante diferente da classificação tradicional, principalmente no que se refere ao tipo de ambiente para o qual foi idealizado. Dessa forma, estudaremos inicialmente as abordagens clássicas de treinamento.

4.1 Treinamento supervisionado

Entende-se por treinamento supervisionado aquele que pode ser realizado com base em amostras de entrada/saída validadas por alguém que possua conhecimento do meio. O diagrama abaixo ilustra esse tipo de treinamento:



Figura 9 – Treinamento supervisionado



Fonte: Haykin, 2011, p. 88.

Observe, na Figura 9, que nesse tipo de treinamento a saída do sistema que está aprendendo é comparada com a saída *correta*. A diferença entre ambas poderá ser utilizada para corrigir os parâmetros do sistema. Veja que tipo de treinamento guarda muita semelhança com o que descrevemos como Aprendizagem por correção de erro, mas, de fato, aquele modelo de aprendizagem é apenas um dos tipos de possíveis de treinamento supervisionado.

Quando em um processo de ML sujeitamos um *dataset* a um algoritmo de aprendizagem de máquina, a aprendizagem será supervisionada se a previsão que desejamos que o algoritmo faça está baseada em dados presentes no *dataset*.

Dito de outra maneira, voltando a nosso já cansado exemplo do corte de árvores, se desejamos a previsibilidade do corte, o treinamento com os dados anteriores será supervisionado. Mas se desejarmos uma informação não



presente nos dados de treinamento (por exemplo, a resistência da madeira obtida por esse corte), o treinamento não será supervisionado, uma vez que não há comparações possíveis com os dados amostrados.

Um algoritmo que aprende de forma supervisionada precisa controlar o erro, é o que fazemos: quando em algoritmos de ML, separamos parte das amostras para medir a acuidade do treinamento. Neste caso, estamos tentando avaliar o quanto o treinamento tornou o algoritmo capaz de generalizar, ou seja, de acertar nas previsões futuras. Observe que, se a generalização não for boa, podemos tentar, na mesma amostra, selecionar outros subconjuntos de treinamento/teste com resultados diferentes. Se os resultados forem muito diferentes, entretanto, pode-se concluir que houve viés na obtenção da amostra, indicando a necessidade de alterar o *dataset*.

4.2 Treinamento não supervisionado

A princípio quando pensamos em um treinamento não supervisionado puro, o aprendizado parece-nos impossível. Imagine um grande conjunto de dados que relacione dados sobre árvores como altura, espessura do tronco, espessura da casca, cor das folhas e densidade da seiva. Suponha que esses dados foram coletados em árvores aleatórias, pertencentes a uma floresta nativa. Seria possível, por um algoritmo de aprendizagem operando somente com esses dados, estimarmos a quantidade de espécies de árvores nessa floresta?

Se você respondeu que sim, acertou. Observou-se que há uma regularidade estatística em fenômenos naturais, ou seja, em nosso exemplo das árvores, uma espécie terá dados próximos em relação às variáveis selecionadas. A espessura da casca, por exemplo, se relacionada à altura da árvore, apresentará uma média diferente para cada espécie. Dito de outra forma, os dados tendem a se auto-organizar. Esse é o motivo pelo qual parte da literatura nomeia o treinamento não supervisionado como aprendizagem auto-organizada. Em nosso exemplo, se encontrarmos quantas categorias ou agrupamentos naturais existem, saberemos, aproximadamente, quantas espécies de árvores existem.

Sobre esse fato, aplicado às redes neurais, Haykin (2011, p. 430) comenta, baseado nas conclusões de Alan Turing, que a repetição de



“interações locais, originalmente aleatórias, entre neurônios vizinhos de uma rede, podem se fundir em estados de ordem global e finalmente, levar a um comportamento coerente na forma de padrões”.

A compreensão da tendência à organização em padrões, no aparente caos dos processos naturais, sempre foi motivo de estudo nas diversas áreas da ciência e não apenas em computação. Em 1976, o Prêmio Nobel de química foi concedido a Prigogine pela aguardada comprovação dessa tendência natural à auto-organização (Di Biasi, 2013).

A observação do fenômeno permitiu afirmar que um sistema originalmente caótico tende a produzir, ao longo do tempo, padrões emergentes estáveis. Aplicando-se esse conceito às redes neurais, pode-se inferir que os parâmetros livres da rede tenderão, se permitirmos que se alterem por si, a gerar a memória de uma rede capaz de compreender, ou generalizar, o processo, inicialmente caótico, a que ela estava sujeita.

Os algoritmos de aprendizagem hebbiana e competitiva são exemplos de aplicação dessa constatação teórica.

TEMA 5 – TREINAMENTO POR REFORÇO – RL

O treinamento por reforço (ou *reinforcement learning*, em inglês) reflete um tipo de aprendizagem em que a evolução do agente é obtida pela análise da reação do meio às suas ações. O agente pode ser previamente treinado ou não.

Ao ser exposto ao meio, o agente, cuja política prevê o treinamento por reforço, interpreta a reação do meio a cada ação e as traduz segundo uma interpretação dicotômica. Uma ação que deve ser repetida é identificada por uma reação positiva do meio, dita recompensa ou reforço. Já uma reação negativa do meio (chamada de *punição*) inibirá futuros comportamentos semelhantes.

O aprendizado por reforço pode ser entendido como um tipo de treinamento não supervisionado. Veja que, de fato, não há um *professor* envolvido, apenas a reação do meio, que pode inclusive ser contraditória, recompensando e punindo a mesma ação em momentos diferentes. Vamos introduzir alguns conceitos sobre esse tipo de treinamento que merecerão ser aprofundados posteriormente para que se possa entender o potencial dessa forma de IA.



5.1 Ambiente estocástico e decisão sequencial

Os ambientes nos quais atuam os sistemas de IA, treinados de forma supervisionada ou auto-organizada, são presumivelmente determinísticos ou episódicos. Vamos entendê-los rapidamente.

Ambientes determinísticos são aqueles que apresentam boa estabilidade e previsibilidade. Nestes ambientes a amostragem dos eventos passados representa um espaço de decisão atemporal, ou seja, coletados os dados de eventos aleatórios, estes balizarão as decisões, sem necessidade de alterações no tempo.

Ambientes de decisão episódica ou instantânea são aqueles nos quais o problema apresentado ao algoritmo é resolvido por uma única decisão, complexa ou não. Os exemplos com os quais trabalhamos neste capítulo são todos episódicos.

Suponha agora que o sucesso da ação do agente no ambiente dependa de uma série de decisões. As estratégias de tratamento para problemas episódicos não nos são úteis neste novo meio de decisões sequenciais.

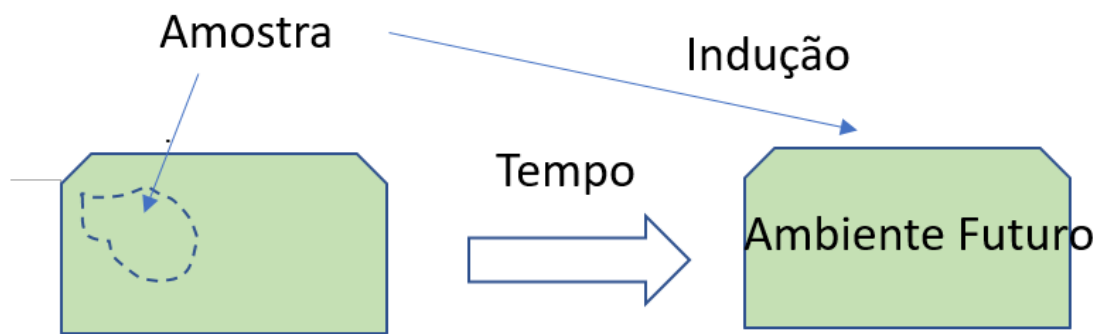
Outra questão a ser enfrentada se refere à característica determinística do meio pressuposto para a decisão episódica. O meio com o qual o agente interage pode não ser estável, ou seja, pode modificar-se com o passar do tempo. De fato, se meditarmos sobre o assunto, concluiremos que os meios determinísticos, estáveis, são um caso particular dos meios instáveis ou estocásticos, que é o caso geral. Naturalmente o tratamento desse caso geral é mais complexo que sua particularização determinística.

Neste ponto você pode estar se perguntando como os métodos estatísticos (estocásticos portanto) de ML podem ser válidos se o ambiente para o qual eles foram pensados é determinístico. Na verdade, a aproximação estatística dos problemas, em ML, permitindo a previsão e descrição dos eventos presentes e passados, tem uma conotação estatística apenas no aspecto de montagem do *dataset*.

Coletamos uma amostra de dados a respeito do fenômeno e descrevemos o todo em função desta parte, a *amostra*. A técnica que permite essa generalização é a *análise estatística*, que só pode ser válida se o ambiente permanecer estável. Essas conclusões são ilustradas a seguir:

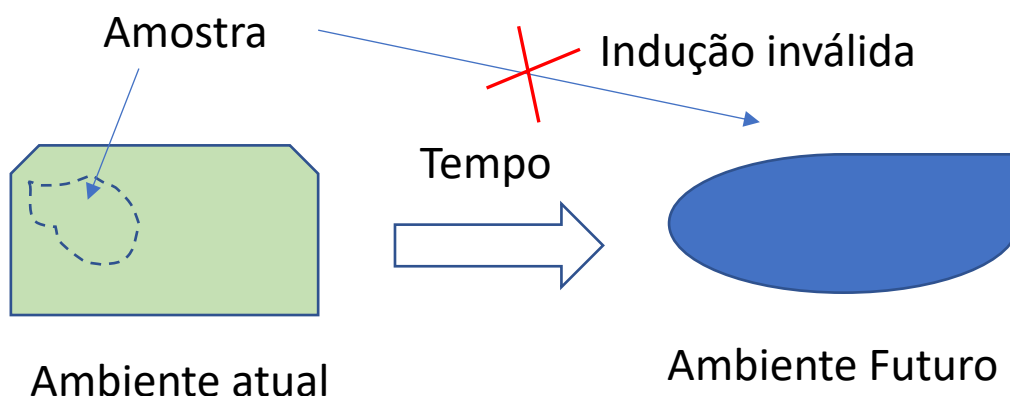


Figura 10 – Previsibilidade em ambiente determinístico



Em um ambiente estocástico, a amostragem pode não nos ser útil como meio de prever a *reação* do meio em um tempo diferente daquele no qual a amostra foi tomada. A Figura 10 ilustra essa situação.

Figura 11 – Previsibilidade em ambiente estocástico



Outra maneira de obter a previsibilidade ou, ao menos, um sinal ponderal de qual a melhor ação a tomar em dado momento precisa ser obtida por outro método que leve em consideração a evolução temporal do meio.

Umas das razões importantes para a instabilidade do meio é a mudança deste frente à ação anterior do próprio agente. De qualquer forma, o agente deve reavaliar o meio a cada decisão tomada. A próxima decisão será então tomada baseada na percepção atual de um meio que está em mutação e, por esse motivo, não alcançará necessariamente seu objetivo. Deve-se então pensar em uma política, para o agente, que analise também a eficiência da série de decisões tomadas para se alcançar dado objetivo. Esse problema foi enfrentado pelo matemático russo Andrei Markov no início do século passado.



5.2 Modelo decisório de Markov

Retomando a questão do meio em mutação, no momento em que o agente realiza uma ação, o meio já terá mudado em relação àquele que motivou a ação. O agente agirá com base em informações de um ambiente que não mais existe, por isso o resultado da ação será, inevitavelmente, diferente do pretendido.

Naturalmente as mudanças do meio tendem a ser gradativas, ou seja, um meio não mudará radicalmente no lapso de tempo entre a análise da decisão e a ação. Assim, mesmo com essa imprevisibilidade do ponto de vista absoluto, os resultados das ações tendem a se aproximar do pretendido, estatisticamente. Podemos imaginar que certas decisões gerarão ações que podem ter maior eficácia que outras. O agente, após a ação, analisa novamente o meio e traduz a eficácia por meio de uma recompensa, positiva ou negativa. Ao longo de uma sequência de decisões, a somatória das recompensas recebidas fornecerá informações sobre a assertividade das decisões do agente. Esse processo descreve rudimentarmente o chamado MDP ou Markov Decision Process.

O MDP se baseia na suposição de Markov, segundo a qual o resultado de uma ação futura pode ser estimado estatisticamente levando-se em consideração um conjunto finito de estados anteriores conhecidos (Norwig, 2013, p. 498)

5.3 Conceito de utilidade e a solução de Bellman

Em um ambiente determinístico treinamos um agente com base na amostragem dos fenômenos do ambiente. Entendemos as variáveis principais que condicionam o sucesso do agente e então treinamos o agente em função de vetores de entrada, compostos por essas variáveis, para os quais a saída, em termos de sucesso ou insucesso do agente, é conhecida. Em ambientes estocásticos, não há essa relação unívoca entre o vetor de entrada e a saída. Um mesmo vetor pode ter mais de uma saída, com probabilidades de ocorrência diferentes.

Podemos imaginar que é possível, entretanto, coletar amostras de pares entrada/saída desde que a ele agreguemos a informação estatística. Vamos propor o exemplo de um robô em movimento em um meio estocástico



bidimensional descritível, para elucidar essa afirmação anterior. Conhecido o vetor de variáveis de entrada, composto pela posição do robô e tipos de obstáculos presentes, o objetivo é deslocar o robô de A para X. Suponha que o movimento de um robô, diante de um obstáculo, em C, desviando para a direita, historicamente, tem 50% de chance de ser bem-sucedido. Desviar para a esquerda tem chance de sucesso de 20% e recuar 30%. A decisão mais assertiva, chamada de *solução* (ou “s”), nesse exemplo, será o desvio à direita.

Como o ambiente está em mutação, essa decisão pode resultar em menor eficiência para alcançar o objetivo. O modelo de Markov prevê que se memorize para essa ação uma recompensa R se a evolução em direção a X for boa ao final do trajeto em função dessa decisão, e uma penalidade P se a decisão não for boa, naturalmente $R > P$. O valor de R e P dependerão de qual danosa ou proveitosa foi a decisão em C.

A utilidade U do desvio ocorrido em C pode ser julgada pela somatória das recompensas deste C até X (Norvig, 2013, p. 566), ou:

$$U_h([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \dots$$

Por esta equação se poderá reavaliar, posteriormente, a assertividade de cada atitude, permitindo reestimar a probabilidade das ações.

Se for possível calcular a utilidade ótima, será possível determinar o melhor caminho, em nosso exemplo, entre A e X. A equação de Bellman nos fornece possíveis soluções para esse problema de otimização, mas esse estudo não pode ser encetado nesse momento inicial, dada a sua complexidade teórica.



5.4 Ambientes de RL

Já percebemos, neste breve estudo de RL, que a inserção do ambiente não determinístico altera perceptivelmente a maneira de se treinar o algoritmo de IA. É de se esperar que o estudo do ambiente, então, seja crucial na definição das estratégias de enfrentamento do problema, mas não basta a mera classificação e análise de ambientes; deve-se também buscar uma parametrização de ambientes hipotéticos que permitam a simulação das funcionalidades do algoritmo. Este viés de estudo se tornou uma área importante. Isso significa que ambientes de simulação, como o *ambiente de Littman*, ganharam importância tão grande quanto dos próprios algoritmos de RL.

FINALIZANDO

Nesta etapa, buscamos construir o conceito de IA, das abordagens e ferramentas existentes, descrevendo algumas das técnicas possíveis de implementação de processos de treinamento para cada linha de IA.



REFERÊNCIAS

COPPIN, B. **Inteligência artificial**. São Paulo: Grupo Gen, 2010

DI BIASE, F. Sistemas auto-organizadores físicos, biológicos, sociais e empresariais. **International Journal of Knowledge Engineering and Management (IJKEM)**, v. 2, n. 2, p. 123-146, 2013.

FACELI, K. et al. **Inteligência artificial** – uma abordagem de aprendizado de máquina. 2. ed. São Paulo: Grupo Gen, 2021.

GORGENS, E. B. et al. Estimação do volume de árvores utilizando redes neurais artificiais. **Revista Árvore**, v. 33, n. 6, p. 1141-1147, 2009.

HAYKIN, S. **Redes neurais**. Pòrto Alegre: Grupo A, 2011.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436-444, 2015. Disponível em: <<https://s3.us-east-2.amazonaws.com/hkg-website-assets/static/pages/files/DeepLearning.pdf>>. Acesso em: 8 abr. 2022.

NORVIG, P. **Inteligência artificial**. 3. ed. São Paulo: Grupo Gen, 2013.

MEDEIROS, L. F. **Redes neurais em Delphi**. 2. ed. Florianópolis: Visualbooks, 2007.